

PS15 - Route Optimization Using a Permutation-Based Genetic Algorithm

Aamna - 2025AB05175

Md M Zaman - 2025AB05168

Rithesh S - 2025AA05655

Sriram TS - 2025AA05094

Vishal R - 2025AA05390

1. Abstract

This document justifies the design of a permutation-based genetic algorithm for the route optimization assignment. It explains why the route is encoded as a permutation, why ordered crossover and swap mutation are suitable, why rank selection is preferred, how each parameter value is chosen, and why the weighted fitness function uses normalized distance and time with a penalty term. It also discusses an alternate NSGA-II formulation and its performance implications.

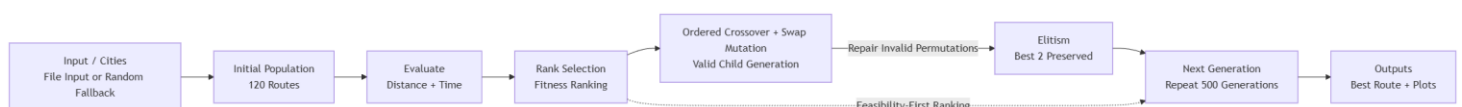
2. Problem formulation

The problem is a constrained route-planning task over 20 cities. A valid solution must visit every city exactly once, return to city 0, keep total distance under **Cmax**, and keep total travel time under **Tmax**. The route is therefore modeled as a permutation, where each chromosome represents a valid city traversal sequence. This representation naturally matches the assignment requirement for a permutation-based Genetic Algorithm (GA) and ensures that the search space remains aligned with valid tours.

The system begins by reading the city coordinates and problem constraints from the input file. An initial population of 120 valid routes is then generated, with each route starting from city 0 and visiting every other city exactly once. Each candidate route is evaluated using total travel distance, derived travel time, and feasibility checks. A normalized fitness function combines distance and time while applying penalties to routes that violate constraints.

The evolutionary process uses **Rank Selection** to choose parent routes, **Ordered Crossover (OX)** to preserve useful route segments while maintaining valid permutations, and **Swap Mutation** to introduce controlled diversity. Invalid routes, if any, are repaired before evaluation. Additionally, the best two solutions are preserved through **elitism** to prevent the loss of high-quality routes.

This cycle of evaluation, selection, crossover, mutation, repair, and elitism is repeated for 500 generations. Finally, the algorithm outputs the best route found, route-evolution snapshots, convergence graphs for distance and time, and comparative results against Random Search and Hill Climbing.



3. Why these modeling choices were selected?

Permutation encoding is the natural representation because the problem is a routing problem, not a classification or continuous optimization task. Any chromosome that is a valid permutation corresponds to

a legal tour, which makes the genetic operators easier to interpret and debug. **City 0** is fixed as the anchor point so that every candidate route starts and ends at the same origin, which makes comparisons between routes meaningful.

The **fitness function** is intentionally normalized. Raw distance and raw time are on different scales, so combining them without normalization would let the larger-magnitude term dominate the score. A 0.5/0.5 weighting keeps the trade-off symmetric, which is consistent with the assignment wording that both time and cost should be optimized together. The **penalty term** is multiplied by 25 so that infeasible routes are still explored early but are strongly discouraged once feasible routes exist.

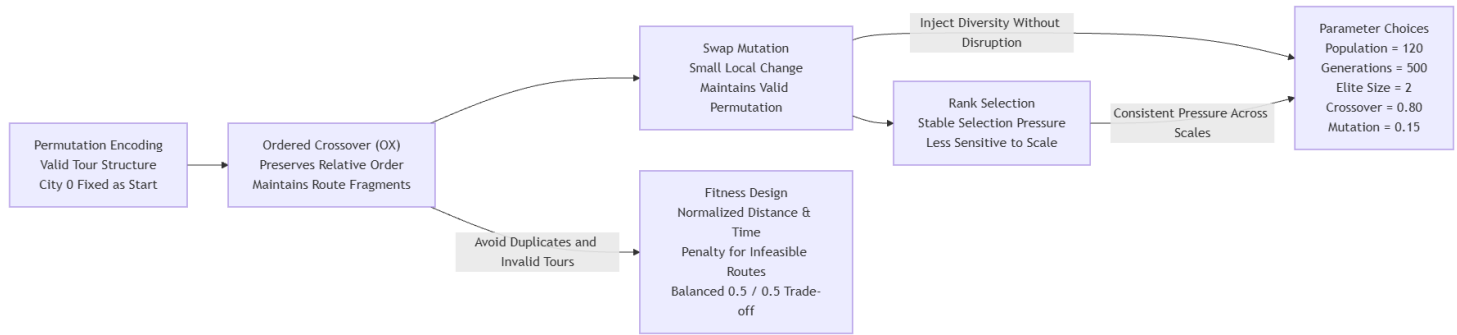
Parameter	Value	Why this value was chosen
Population size	120	Large enough to maintain diversity in a 20-city permutation space, yet still practical for 500 generations.
Generations	500	Matches the assignment requirement and provides enough iterations for visible convergence without excessive runtime.
Elite size	2	Protects the best routes while preserving diversity; larger elite fractions can prematurely freeze the population.
Crossover rate	0.80	Makes crossover the primary search operator while leaving room for direct inheritance and mutation-driven variation.
Mutation rate	0.15	High enough to inject diversity, but low enough not to destroy useful building blocks too often.
Fitness weights	0.5 / 0.5	Gives equal importance to cost and time, which is the fairest default when no priority is specified.
Penalty weight	25	Strong enough to prioritize feasibility without making the search collapse too early to a narrow region.
Speed assumption	8 units/hour	Transforms geometric distance into time on the same order of magnitude as $T_{\max}$, keeping the two objectives comparable.

4. Why the specific algorithms were chosen?

Rank selection was chosen because it is less sensitive to raw fitness scale than roulette wheel selection. In this problem, routes can vary widely in distance and time, so a rank-based mechanism produces steadier selection pressure and avoids letting one strong individual dominate early. It is also easier to justify in a strict review because the selection probability depends on relative quality rather than arbitrary score magnitude.

Ordered crossover (OX) is the correct crossover for permutation problems because it preserves city order and prevents duplicates. Classical one-point crossover can easily break permutation validity, while OX keeps route fragments intact and reduces the amount of repair needed.

Swap mutation was selected because it is a minimal local change that still preserves a valid permutation; it is simpler and safer than stronger mutations that can disrupt good partial tours. Elitism with two individuals is retained because it protects progress while keeping the exploration budget high.



5. Alternate modeling approach and performance implications?

An alternative formulation is **NSGA-II**, where distance and time are modeled as two separate objectives instead of being merged into one weighted fitness. NSGA-II would return a Pareto front of trade-off routes and would be academically stronger if the task were explicitly multi-objective. However, it would increase computational overhead because non-dominated sorting and crowding-distance calculations must be performed every generation. It also produces a larger solution set, which is more complex to present within a short assignment report. For this assignment, the weighted-sum GA is the better engineering choice because it is simpler, faster, and easier to explain while still meeting the required constraints.

6. Program Execution and Outputs

The program is executed using Python 3 and supports command-line arguments for specifying input and output file locations. The input file contains city coordinates and constraint values ($C_{\max}$ and $T_{\max}$). If no input file is supplied, a random set of 20 cities is generated for testing purposes. Example execution:

```
python route_optimization_ps15.py -input inputPS15.txt -output outputPS15.txt
```

During execution, the system generates an initial population of routes, evaluates their fitness, and iteratively applies Rank Selection, Ordered Crossover (OX), Swap Mutation, and Elitism over 500 generations to identify an optimal route.

The program produces both textual and graphical outputs, including the best route sequence, total distance, travel time, fitness score, and feasibility status. Additionally, it generates route visualizations, route-evolution snapshots, distance and time convergence plots, and a performance comparison against Random Search and Hill Climbing. These outputs provide both quantitative and visual insights into the optimization process and final solution quality.

7. Experimental observations

On the recorded run, the genetic algorithm improved from a starting distance of 809.45 to 441.90 and travel time from 101.18 to 55.24. Random search converged more slowly, while hill climbing produced a lower route cost on the same seed in this run. This is a useful result rather than a failure: it shows that local search can be strong on a favorable initialization, but the genetic algorithm remains the more general search framework because it explores multiple regions of the search space simultaneously and does not depend on a single starting point.

Method	Distance	Time	Fitness	Feasible?
Genetic Algorithm	441.90	55.24	0.3850	Yes
Random Search	660.76	82.60	0.5757	Yes
Hill Climbing	360.68	45.09	0.3142	Yes

8. Conclusion

The design follows the assignment requirements closely: it uses permutation encoding, ordered crossover, swap mutation, rank selection, random-city fallback, 500 generations, and comparison baselines. The parameter choices are justified by the need to balance diversity, convergence speed, feasibility pressure, and computational cost. The resulting solution is compact, modular, and suitable for a strict evaluation.

Github link: <https://github.com/dreamywarrior/route-optimization-genetic-algorithm/tree/final-version>